MotoTrack Final Production-Readiness Review

Focused go-live audit of the updated reservation, OTP, stock, and authorization flows

Review date	October 7, 2026
Review type	Static code review of mototrack_source_for_chatgpt_v2.zip
Scope	Only remaining HIGH and CRITICAL go-live blockers
Overall verdict	NO-GO until critical state/security issues are corrected

NO-GO: Fix items 1-4 before launch. Items 5-6 should also be corrected before exposing the site to real customers.

Executive Summary

The V2 codebase contains the major fixes from the previous audit: the public debug-log route is removed; OTPs are generated with random_int(), hashed, and throttled; OTP enforcement is globally applied; checkout reservations begin as pending with zero amount tendered; and the hourly reservation-expiration command is registered.

However, six remaining high/critical issues were identified. The most important risks are exposed environment secrets, a checkout transaction bug, the absence of a reliable pending-to-completed reservation transition, and replayable stock-changing actions that can inflate inventory.

All 246 PHP files passed PHP syntax checking. The supplied archive did not contain composer.json or artisan, so the Laravel/Pest test suite could not be executed from this package.

1. Populated .env file with live-looking secrets and APP_DEBUG=true

CRITICAL

The archive contains a populated .env containing database credentials, Queen SMTP credentials, Google OAuth secret, reCAPTCHA secret, Xendit credentials, SMS/API credentials, Reverb secrets, and the Laravel APP_KEY. Because the file has now been packaged/shared, treat those credentials as exposed.

Required before launch

- Remove .env from Git and all release/source archives.
- Ensure .gitignore excludes .env.
- Rotate every exposed credential, password, key, and token.
- Set APP_ENV=production and APP_DEBUG=false on the deployed server.
- Remove public exception details from PublicBrowseController.php; log/report internally and return a generic 500 response.

2. CommitOrder can lose the $status variable inside the transaction closure

CRITICAL

app/Actions/CommitOrder.php passes $status into execute(), but the DB::transaction closure does not capture $status in its use (...) list while later assigning 'status' => $status. This can break checkout/POS transaction creation or produce an invalid status.

```
DB::transaction(function () use (
    $lines,
    $transactionReference,
    $customerId,
    $stockErrorPrefix,
    $tenderFor,
    $changeFor,
    $dispatchLowStockAlert,
    $status
) {
```

Required before launch

- Add $status to the closure's use (...) capture list.
- Add an automated test proving online checkout creates pending rows.
- Add an automated test proving POS creates completed rows.

3. No reliable pending -> completed reservation transition

HIGH

Online checkout correctly creates pending Sale rows, but there is no confirmed owner action that atomically marks the reservation completed when the customer pays and picks up the item. A reservation can therefore remain pending after physical payment and later be expired by the hourly command, restoring stock that was already released to the customer.

Required before launch

- Add an owner-only Paid & Picked Up / Complete action.
- Transition the entire transaction_reference atomically inside DB::transaction().
- Lock pending reservation rows with lockForUpdate().
- Re-check the rows are still pending before completion.
- Set status=completed, amount_tendered, and picked_up_at together.
- Ensure expiration and completion contend on the same row locks so only one transition wins.

4. Replayable stock-changing actions can inflate inventory

HIGH

Several state-changing actions are not fully idempotent. AdminPreOrderController::complete() can increment stock more than once because the PreOrder itself is not locked/re-validated. RefundController::approve() lacks a transaction, lock, and pending-state guard. CustomerOrderController::requestRefund() can race without a database uniqueness invariant. SaleController::destroy() restores stock regardless of current sale state, so deleting an already-cancelled reservation can restore stock twice.

Required before launch

- Lock the state record itself before any stock-changing transition.
- Re-check the allowed source state under the lock.
- Make complete/cancel/refund/expire transitions idempotent.
- Add a database-level uniqueness/invariant preventing multiple active refunds for one sale.
- Prevent SaleController::destroy() from restoring stock for states that were already restored.

5. Customer UI still contains old online-payment/downpayment instructions

HIGH

The Pure Reservation migration is incomplete in the frontend. checkout.blade.php still contains GCash/payment language, QR/payment sections, 'Total Payment Amount (100%)', and text saying full payment is required on checkout. welcome.blade.php still advertises GCash payment. The pre-order admin flow also retains legacy pay/markPaid/downpayment concepts.

Required before launch

- Change checkout language to Reserve / Pay on Pickup.
- Remove QR/full-payment controls from customer checkout.
- Remove all remaining 50% downpayment/full-payment wording.
- Remove or disable legacy preorder pay/markPaid actions from the active workflow.
- Update customer-facing Terms/FAQ/welcome content so it matches backend behavior exactly.

6. Database users still default to owner

HIGH

The role migration still defines $table->string('role')->default('owner'). Current registration paths explicitly set customer, but the database default remains unsafe: any import, seeder, maintenance script, or future User::create() call that omits role can silently create an administrator.

Required before launch

- Change the live database default to customer, or remove the default entirely.
- If the migration has already run, create a new migration that alters the existing default instead of editing only historical migration files.

What Passed the Final Review

No remaining high/critical customer-order IDOR or obvious SQL injection path was identified in the reviewed code. Receipt queries are customer-scoped, customer cancellation/refund ownership checks are present, owner routes are protected, OTP middleware is globally enforced, OTP hashing/randomness are corrected, and the expiration command re-checks pending state while holding the Sale lock.

Final Launch Gate

Priority	Gate	Required result
P0	Secrets and production config	Rotate exposed secrets; APP_DEBUG=false; .env excluded
P0	CommitOrder status bug	Pending and completed creation covered by tests
P0	Reservation completion	Atomic pending -> completed flow with row locks
P0	Inventory idempotency	No transition can restore/increment stock twice
P1	Pure Reservation UI	No active online-payment/downpayment wording or actions
P1	Role default	No database default that grants owner privileges
Operational	Laravel scheduler	schedule:run every minute or equivalent worker is active

Operational reminder: defining Schedule::command('reservations:expire')->hourly() does not execute it by itself. The production server must invoke Laravel's scheduler (typically schedule:run every minute) or use an equivalent persistent scheduler/worker.

NO-GO: Fix items 1-4 before launch. Items 5-6 should also be corrected before exposing the site to real customers.

Prepared from the final static review of mototrack_source_for_chatgpt_v2.zip. This report intentionally includes only HIGH and CRITICAL remaining go-live issues.